

OS LAB 4 TASKS(24K-1033)

Q1

CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <sys/types.h>

void executeCommand()
{
    execlp("bash", "bash", "-c", "ls -l");
}

void main()
{
    pid_t pid=0;
    pid=fork();
    if(pid==0) executeCommand();
    wait(0);
    printf("Child process has finished executing");
    return;
}
```

OUTPUT

```
total 20
-rwxr-xr-x 1 runner4 runner4 16128 Feb 17 18:32 a.out
-rw-r--r-- 1 runner4 runner4   376 Feb 17 18:32 main.c
Child process has finished executing
```

Q2

CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <sys/types.h>

void main()
{
    pid_t pid=0;
    pid=fork();
    pid=fork();
    if(pid==0)
    {
        printf("Child: process-id:%d
parent-process-id:%d",getpid(),getppid());
    }
    else
    {
        wait(NULL);
        printf("Both Child processes have finished\n");
    };
    return;
}
```

OUTPUT

```
Child: process-id:9078  parent-process-id:9073  
Child: process-id:9079  parent-process-id:9077  
Both Child processes have finished
```

Q3

ANSWER

- a. There are 4 children and 1 parent meaning a total of 5 processes.**
- b. 5 times**
- c. Then it will be 8 processes**
- d. By all processes as long as loop continues**

Q4

Answer

- a. One process is created**
- b. Child process runs pwd**
- c. No**
- d. Twice**
- e. no**

Q5

Answer

- a. 10 (the exit status)**

- b. Wait doesn't return anything but it alters the value pointed to by the pointer parameter.**
- c. The parent won't wait for the child process to end and will immediately start printing.**
- d. Then 0 will be printed**

Q6

Answer

- a. Parent prints first due to delay (by sleep function)**
- b. Sleep is used to delay the child process by 2 milliseconds**
- c. It returns the process id of the parent process**
- d. yes(that scenario will create an orphaned process)**

Q7

Answer

- a. 2 seconds**
- b. "Time up!" Is printed**
- c. Program will either be stuck in infinite loop or it will prematurely exit the process or a default os-specific signal will be created.**
- d. To do an infinite loop, so that the main process waits until the alarm is activated.**